

BFS search on permutations with set constraints

10/09/2026 02:51:09

<https://claude.ai/share/34f8bf9d-50aa-44eb-8f82-62b55a5b83c9>

Fix some $n \geq 1$. For x, y in $[n]$, let $\tau_{x,y}$ be the...

I'll write a BFS script that explores this state space. Let me think through...

Yeah assume the root is always $(\emptyset, \{1\}, id)$

Updated — the default root is now $(\emptyset, \{1\}, id)$. (The printed banner sti...

I want to understand the structure of this graph and try to figure out if al...

Short answer: **no** — not all n -cycles are reached. But the leaf...

you misunderstood my question, by n -cycles I meant cycles of length...

Ah, my interpretation was actually the same — `is_n_cycle` checked...

show me all the nodes in the tree for $n=3$

For $n = 3$ the tree has 4 nodes: depth 0: $(L=\emptyset, S=\{1\}, \pi=[1,2,3])$...

can you run it again with root $([n], \emptyset, id)$ for $n=3$ and show m...

Tree for root $(\{3\}, \emptyset, id)$ with $n=3$: depth 0: $(L=\{3\}, S=\emptyset, \pi=...$

something is wrong, L and S must always be disjoint by the given...

You're right. The rule as literally written in the original message says S ...

sure check for such a condition, also check the counts for the root $(...$

Let me check one more angle — whether the two roots are related by...

please search for structural predicates

Good — I found some real structural signals. Let me dig deeper on the...

Are there two distinct cycles with the same S set?

Yes, lots. Since $L \cup S = [n]$ at n -cycle leaves, knowing S also...

How many cycles are there per S ?

Very interesting structure emerges. Let me tally by $|S|$: Hmm wait — fo...

no. I want to prove that no two leaves reach the same cycle, can you hel...

Let me first confirm the conjecture computationally across a wider range...

what are the number of leaves for n -trees for roots $(\emptyset, \{i\}, ...)$

Here are the leaf counts for root $(\emptyset, \{i\}, id)$, with i varying across...

do the total sums equal the total number of n -cycles

Let me check — the row sums are all leaves, but you might mean just n ...
yes find the first missing n -cycle

The first missing n -cycle is for $n = 4$: the cycle $(1, 3, 4, 2)$ in cycl...
update the algorithm so that we have a third way to add a child, pick...

Yes — the invariant **still holds, and in a cleaner form**. With Rule C...
now check if all the leaves correspond to n -cycles

Not all leaves are n -cycles. Full breakdown: $|n|$ leaves $|n$ -cycles $|$ cycle...
update rule C so that instead of picking x, y not in $L \cup S$, we just...

The update **breaks everything**: $|n|$ nodes $|$ leaves $|$ invariant holds? ...
replace all the rules A,B,C with a single rule, pick x not in L , y not...

Beautiful — this unified rule gives a much cleaner structure: $|n|$ nodes $|$...
how do i prove the total number of nodes is n !

Here's a proof sketch. The key pieces are already in hand — you have th...
write out a latex document doing the following * defining the latest tree...

Let me first verify computationally which choices of witness (x, y) alwa...
you can induct on the length of the permutation (number of inversions),...

The induction on $\text{inv}(\pi)$ replaces the displacement argument. The...
recall the definition of good and increasing reps from another chat

I don't have memory of other chats — each conversation starts fresh for...
A transposition σ_{ij} is a permutation that just swaps i and j . A...

Formalization Fix $n \geq 1$. A transposition on $[n]$ is specified by...
An increasing representation of a permutation π is a composition of...

Formalization Let $\ell : S_n \rightarrow \mathbb{Z}_{\geq 0}$ denote the...
Let me redefine "good". A collection of transpositions $C = \{\sigma_{ij}\}$ is...

Formalization Let C be a finite set of distinct transpositions on $[n]$...
A good representation of a permutation is a composition of transposition...

Formalization **Definition.** Let $\pi \in S_n$. A **good**...
show that every path in the tree gives a good and increasing...

Transpositions along paths are always distinct. Here's the proof, written ...



Fix some $n \geq 1$. For x, y in $[n]$, let $\tau_{x,y}$ be the transposition of x and y . Write a BFS search script that performs the following. . Every node explored is going to be of the form (L, S, π) where π is a permutation of $[n]$ and L, S are sets. For a given node (L, S, π) , we get the children nodes by doing the following. For any $x \in S$ and $y \notin L \cup S$ such that $\pi(y) = y$, $\pi(x) < y$ and $x < y$, we have a child node $(L \cup \{y\}, S, \pi \circ \tau_{x,y})$. For any $x \in L$ and $y \notin L \cup S$ such that $\pi(y) = y$, $\pi(x) > y$ and $x > y$, we have a child node $(L, S \cup \{x\}, \pi \circ \tau_{x,y})$.

4/14/2026, 3:47:53 AM



I'll write a BFS script that explores this state space. Let me think through the structure first, then implement it.

Created file ▾

📄 BFS script for the permutation state space

🔄 Done

Let me test it for small **n** to make sure the rules are firing correctly:

Ran command ▾

📄 Tool: bash_tool

🔄 Done

Starting from **($\emptyset, \emptyset, \text{id}$)** both rules are vacuous (they require **$x \in S$** or **$x \in L$**), so no children are generated — the BFS just stays at the root. That's a correct consequence of the rules as stated, but probably not what you want to explore. Let me also run it with some non-trivial starting nodes so you can see the machinery working:

Created file, ran command ▾

📄 Demo with seeded starting nodes

📄 Tool: bash_tool

🔄 Done

Working as specified.

The script is in **/home/claude/bfs.py**. Key pieces:

- **State:** (L, S, π) with L, S as `frozenset`s (so they're hashable for the visited set) and π as a tuple where $\pi[i-1] = \pi(i)$.
- `apply_transposition_after( $\pi$ ,  $x, y$ )` computes $\pi \circ \tau_{\{x,y\}}$, i.e. $(\pi \circ \tau)(x) = \pi(y)$, $(\pi \circ \tau)(y) = \pi(x)$.
- `children( $node, n$ )` implements both rules exactly as stated, requiring $y \notin L \cup S$, $\pi(y) = y$, plus the comparison conditions.
- `bfs( $n, start\_nodes=...$ )` does standard BFS with a visited dict recording $(depth, parent)$ so you can reconstruct paths if needed.

One thing worth flagging: starting from $(\emptyset, \emptyset, id)$ the BFS terminates immediately because Rule A needs some $x \in S$ and Rule B needs some $x \in L$. Both rules quantify over a non-empty set, so the empty-empty root is a sink. You'll want to seed with something like $(\emptyset, \{x_0\}, id)$ or $(\{x_0\}, \emptyset, id)$ — `demo.py` shows both. If you actually wanted the empty sets to also admit a "bootstrap" child, let me know and I'll adjust the rules.

Presented file ▾

📄 Presenting file(s)...

✓ Done

 bfs
Py



 demo
Py



4/14/2026, 3:48:49 AM



Yeah assume the root is always $(\emptyset, \{1\}, id)$

4/14/2026, 3:52:17 AM



Edited 2 files, ran command ▾

🔗 Tool: str_replace

🔗 Tool: str_replace

📄 Tool: bash_tool

✓ Done

Updated — the default root is now $(\emptyset, \{1\}, id)$. (The printed banner still says "empty, empty"; let me fix that cosmetic bit too.)

Edited file ▾

🔗 Tool: str_replace

✓ Done

A few observations from the runs:

n	total nodes	max depth	distinct perms
4	9	3	9
5	25	4	25
6	91	5	88

For $n=4, 5$ every reachable node has a distinct permutation, but at $n=6$ that breaks (91 nodes, only 88 perms) — so different (L, S) pairs can share a π . Also the depth- d counts for $n=6$ go $1, 5, 10, 20, 30, 25$ — not an obvious closed form, let me know if you want me to dig into the structure.

Presented file ▾

📄 Presenting file(s)...

✓ Done

📖 bfs
Py



4/14/2026, 3:52:41 AM



I want to understand the structure of this graph and try to figure out if all n -cycles are reached at the leafs of an n -tree

4/14/2026, 3:55:10 AM



Created file, ran command ▾

- 📄 Analyze leaf structure and n-cycle coverage
- 📁 Tool: bash_tool
- 👉 Done

Short answer: **no** — not all n -cycles are reached. But the leaf structure has a very clean shape. Here's what the data shows:

n-cycle coverage

n	$(n-1)!$ total n-cycles	n-cycles reached	fraction
3	2	1	1/2
4	6	2	1/3
5	24	6	1/4
6	120	22	$\sim 1/5.4$
7	720	102	$\sim 1/7$

So the reached-n-cycle count goes 1, 2, 6, 22, 102 — definitely not all of them, and the fraction shrinks fast.

The clean facts

1. Every n -cycle that appears in the tree is a leaf. In all runs, $\text{n-cycles anywhere} == \text{n-cycles at leaves}$. This is structural: once π is an n -cycle there are no fixed points, so the $\pi(y)=y$ condition kills every child rule.

2. Leaves at depth d always have cycle type $(d+1, 1, \dots, 1)$. Look at the depth/cycle-type correspondence for $n=7$:

depth of leaf	cycle type	count
1	$(2, 1^5)$	1
2	$(3, 1^4)$	1
3	$(4, 1^3)$	5
4	$(5, 1^2)$	13
5	$(6, 1)$	31
6	(7)	133

Same pattern for every n . So the tree grows a single nontrivial cycle, extending it by one element per depth level, and each leaf is "stuck" because the nontrivial cycle has consumed all the fixed points it can reach. Max depth is always $n-1$.

3. The leaf cycle-type counts form sequences:

- n -cycle leaves: $1, 2, 6, 22, 102$ ($n=3..7$)
- $(n-1, 1)$ -leaves: $1, 3, 8, 31$ ($n=4..7$)
- $(n-2, 1, 1)$ -leaves: $1, 4, 13$ ($n=5..7$)

Would you like me to (a) reconstruct a witness path for a specific n -cycle that *is* reached (so you can see what the sequence of moves looks like), (b) characterize which n -cycles are reachable vs not (one-line description in cycle notation), or (c) check whether these leaf-count sequences match known OEIS entries?

4/14/2026, 3:55:59 AM



you misunderstood my question, by n -cycles I meant cycles of length n

4/14/2026, 3:57:35 AM



Ah, my interpretation was actually the same — `is_n_cycle` checked whether π is a single cycle of length n (i.e., cycle type $(n,)$, which in `s_n` means a single cycle of length n). So the finding stands: **not all cycles of length n appear** — only `1, 2, 6, 22, 102` of them for $n = 3, 4, 5, 6, 7$ (out of $(n-1)! = 2, 6, 24, 120, 720$).

But there's a subtlety worth flagging that I glossed over, which might be what you were sensing. The tree has more **leaf nodes** at maximum depth than **distinct n -cycle permutations** — different `(L,S)` labels can sit over the same π :

n	leaves at max depth $n-1$	distinct π among them
4	2	2
5	6	6
6	25	22
7	133	102

Starting at `n=6`, some n -cycle appears as the π component of multiple leaves. Let me pull up a concrete example:

Created file, ran command ▾

- Find n -cycles that appear at multiple leaves
- Tool: `bash_tool`
- Done

So the situation for $n = 7$:

- Total cycles of length 7 in `s_7`: **720**
- Distinct such cycles reached as some leaf's π : **102**

- Leaf nodes whose π is a 7-cycle: **133** (28 of those 102 cycles are reached via 2 different **(L, S)** paths)

If the question is really "do all 720 show up somewhere in the tree" — no, only about 14% do. If you want, I can characterize *which* n-cycles are reachable — e.g., write each reached n-cycle in cycle notation **(1, a₂, a₃, ..., a_n)** (always starting with 1, since $1 \in S$ forces 1 to be in the nontrivial cycle) and look for a pattern on the sequences **(a₂, ..., a_n)**. Want me to do that?

4/14/2026, 3:58:37 AM



show me all the nodes in the tree for $n = 3$

4/14/2026, 3:59:59 AM



For $n = 3$ the tree has 4 nodes:

```

depth 0:  (L=∅,      S={1},  π=[1,2,3])           ← root
          |
          └─ Rule A, x=1, y=2:
depth 1:  |  (L={2},    S={1},  π=[2,1,3])
          |  |
          |  └─ Rule A, x=1, y=3:
depth 2:  |  (L={2,3}, S={1},  π=[3,1,2])           ← leaf (3-
cycle)
          |
          └─ Rule A, x=1, y=3:
depth 1:  |  (L={3},    S={1},  π=[3,2,1])           ← leaf
(transposition)

```

Step-by-step:

Root $(\emptyset, \{1\}, [1,2,3])$. Only Rule A can fire (L is empty). With $x=1$, $\pi(1)=1$, the fixed-point y needs $x < y$ and $\pi(x) < y$, so both $y=2$ and $y=3$ work.

Node $(\{2\}, \{1\}, [2,1,3])$ — reached by swapping $1 \leftrightarrow 2$. Rule A with $x=1$, $\pi(1)=2$: need $y > 2$ and $\pi(x)=2 < y$, so $y=3$ works. Rule B needs $x=2 \in L$ with $\pi(2)=1 > y$ and $x=2 > y$, so $y=1$, but $1 \notin L \cup S$. Dead.

Node $(\{3\}, \{1\}, [3,2,1])$ — reached by swapping $1 \leftrightarrow 3$. Rule A: $x=1$, $\pi(1)=3$, need $y > 3$. None. Rule B: $x=3$, $\pi(3)=1$, need $y < 3$ and $y < 1$. None. **Leaf**.

Node $(\{2,3\}, \{1\}, [3,1,2])$ — 3-cycle $(1\ 3\ 2)$. No fixed points remain outside $L \cup S = \{1,2,3\}$, so neither rule fires. **Leaf**.

Both leaves sit on the "spine" of the tree — one at depth 1 (the transposition $(1\ 3)$) and one at depth 2 (the 3-cycle $(1\ 3\ 2)$). The other 3-cycle in S_3 , namely $(1\ 2\ 3) = [2,3,1]$, is never reached.

4/14/2026, 4:00:21 AM



can you run it again with root $(\{n\}, \emptyset, id)$ for $n = 3$ and show me all the sets

4/14/2026, 4:04:04 AM



Ran command ▾

Tool: bash_tool

Done

Tree for root $(\{3\}, \emptyset, id)$ with $n=3$:

```

depth 0:  (L={3}, S=∅,    π=[1,2,3])           ← root
          |
          └─ Rule B, x=3, y=1  (swap 1↔3):
depth 1:  |  (L={3}, S={3},    π=[3,2,1])       ← leaf
          |
          └─ Rule B, x=3, y=2  (swap 2↔3):
depth 1:  (L={3}, S={3},    π=[1,3,2])
          |
          └─ Rule B, x=3, y=1  (swap 1↔3):
depth 2:  (L={3}, S={1,3}, π=[2,3,1])         ← leaf

```

Quick walkthrough:

Root $(\{3\}, \emptyset, [1,2,3])$. S is empty so Rule A is dead. Rule B with $x=3$: need $y<3$, $\pi(y)=y$, $\pi(x)=3>y$. Both $y=1$ and $y=2$ qualify.

$(\{3\}, \{3\}, [3,2,1])$ — after swapping $1 \leftrightarrow 3$. $\pi(3)=1$. Rule A needs $y>x=3$: none. Rule B needs $y<\pi(x)=1$: none. **Leaf**.

$(\{3\}, \{3\}, [1,3,2])$ — after swapping $2 \leftrightarrow 3$. Only remaining fixed point outside $L \cup S = \{3\}$ is $y=1$. Rule A needs $x=3<y$: fails. Rule B with $x=3$, $\pi(3)=2$: needs $y<2$ and $y<3$, so $y=1$. Swap $1 \leftrightarrow 3$ in π gives $[2,3,1]$; adds $x=3$ to S (already there — S stays $\{3\}$ conceptually, but the rule adds $x=1$? let me recheck).

Actually let me double-check that last step — the code says the child's S is $\{1,3\}$:

Ran command ▾

Tool: bash_tool

Done

Right — Rule B adds x (not y) to S , and $x=3$ was already in S , so S stays $\{3\}$. I misread the earlier output. Let me redraw cleanly:

```
depth 0:  (L={3}, S=∅,   π=[1,2,3])           ← root
          |
          └─ Rule B, x=3, y=1  (swap 1↔3, add x=3 to S):
depth 1:  |  (L={3}, S={3}, π=[3,2,1])         ← leaf
          |
          └─ Rule B, x=3, y=2  (swap 2↔3, add x=3 to S):
depth 1:  |  (L={3}, S={3}, π=[1,3,2])
          |
          └─ Rule B, x=3, y=1  (swap 1↔3, add x=3 to S):
depth 2:  |  (L={3}, S={3}, π=[2,3,1])         ← leaf
```

4 nodes, 2 leaves. The two leaves are exactly the two 3-cycles of S_3 : $[3,2,1]$ — wait, $[3,2,1]$ is the transposition $(1\ 3)$, not a 3-cycle. And $[2,3,1]$ is the 3-cycle $(1\ 2\ 3)$. So this root reaches $(1\ 2\ 3)$ (the one the other root missed) but its depth-1 leaf is a transposition, not a 3-cycle.

Notice the symmetry with the previous tree (root $(\emptyset, \{1\}, \text{id})$): that one reached the 3-cycle $(1\ 3\ 2) = [3,1,2]$, this one reaches $(1\ 2\ 3) = [2,3,1]$ — the two trees together cover both 3-cycles.

4/14/2026, 4:04:46 AM



something is wrong, L and S must always be disjoint by the given procedure

4/14/2026, 4:06:21 AM



You're right. The rule as literally written in the original message says $S \cup \{x\}$ for Rule B, but $x \in L$, which would break disjointness. By symmetry with Rule A (which adds the fixed point y to L), Rule B should add the fixed point y to S . That's almost certainly a typo in the problem statement. Let me fix the code:

Edited file, ran command ▾

🔗 Tool: str_replace

📁 Tool: bash_tool

👍 Done

Fixed. Now disjointness holds in both trees. The corrected tree for $n=3$, root $(\{3\}, \emptyset, id)$:

```
depth 0:  (L={3}, S=∅,      π=[1,2,3])
          |
          └─ Rule B, x=3, y=1 (swap, add y=1 to S):
depth 1:  |  (L={3}, S={1},   π=[3,2,1])           ← leaf
          |
          └─ Rule B, x=3, y=2 (swap, add y=2 to S):
depth 1:  |  (L={3}, S={2},   π=[1,3,2])
          |
          └─ Rule B, x=3, y=1 (swap, add y=1 to S):
depth 2:  |  (L={3}, S={1,2}, π=[2,3,1])           ← leaf
```

And the re-run for root $(\emptyset, \{1\}, id)$ is structurally the same as before (Rule A never violated disjointness, so that tree is unchanged).

Let me also redo the earlier coverage analysis since Rule B's behavior changed:

Ran command ▾

📁 Tool: bash_tool

👍 Done

Interesting — with the fix, the n-cycle leaf counts become `1, 2, 6, 24, 120` for `n = 3..7`. That's exactly `(n-2)!`. Let me verify these are all distinct permutations:

Ran command ▾

📄 Tool: bash_tool

👍 Done

Clean pattern:

n	n-cycles reached	<code>(n-2)!</code>	<code>(n-1)!</code> total
3	1	1	2
4	2	2	6
5	6	6	24
6	24	24	120
7	120	120	720
8	720	720	5040

So with the corrected rules:

- **Every n-cycle reached is distinct** (no duplicates across leaves — the earlier duplication was an artifact of the buggy rule).
- **Exactly `(n-2)!` distinct n-cycles are reached**, a fraction `1/(n-1)` of the total `(n-1)!`.

That `1/(n-1)` fraction strongly suggests a conserved quantity picking out a specific coset. Natural guess: in cycle notation starting at 1, the reached n-cycles might all share the same value for some fixed position (e.g. `n` always comes right before 1, or `π(1)=n` always, or similar). Want me to check?

4/14/2026, 4:07:19 AM



sure check for such a condition, also check the counts for the root $(\{n\}, \emptyset, id)$ and see if that covers the remaining cycles

4/14/2026, 4:09:34 AM



Created file, ran command ▾

- 📄 Check invariants and complementary root coverage
- 📄 Tool: bash_tool
- 📄 Done

Let me check one more angle — whether the two roots are related by permutation inversion, and examine the overlap:

Ran 2 commands ▾

- 📄 Tool: bash_tool
- 📄 Tool: bash_tool
- 📄 Done

Nice — the **order-reversing conjugation** $i \mapsto n+1-i$ maps tree A exactly to tree B. This is the structural symmetry of the problem: Rules A and B are swapped under this involution, and **L** and **S** swap roles. So the two trees are isomorphic copies of each other, and asking "do they together cover all n-cycles" is like asking whether this symmetry is a bijection onto a set preserved by it.

Now let me look for the actual invariant characterizing reached cycles. I'll look at each cycle written as $(1, a_2, \dots, a_n)$ and check: does **n** appear in a constrained position? Does $\pi(1)$ have a pattern?

Ran command ▾

- 📄 Tool: bash_tool
- 📄 Done

I've dug around and couldn't find a single-condition invariant (nothing like " $\pi(1) = n$ always" or "**n** at a fixed position"). Here's the summary:

Findings for root $(\emptyset, \{1\}, id)$, $n=6$:

- By $\pi(1)$ value among reached cycles:

- $\pi(1) = 4$: 1 cycle
- $\pi(1) = 5$: 6 cycles
- $\pi(1) = 6$: 17 cycles
- By $\pi(1)$ value among **unreached** cycles: $\pi(1) \in \{2, 3\}$ for all of them, plus some with $\pi(1) \in \{4, 5, 6\}$.

So reached $\Rightarrow \pi(1) \geq \lceil (n+2)/2 \rceil$ is a necessary condition, but not sufficient.

Coverage of the two complementary roots:

n	root A reaches	root B reaches	A $\cap$ B	A $\cup$ B	$(n-1)!$ total	union/total
3	1	1	0	2	2	100%
4	2	2	0	4	6	67%
5	6	6	0	12	24	50%
6	24	24	1	47	120	39%
7	120	120	6	234	720	33%
8	720	720	40	1400	5040	28%

Structural facts:

1. **The two trees are isomorphic** via the order-reversing conjugation $\pi \mapsto (i \mapsto n+1-\pi(n+1-i))$. This exchanges Rules A and B, and swaps $L \leftrightarrow S$.
2. **They don't together cover all n-cycles** — the fraction covered shrinks toward 0 as n grows. And they're not disjoint: for $n \geq 6$ some cycles appear in both.
3. Each tree reaches exactly $(n-2)!$ n-cycles, which is $1/(n-1)$ of all n-cycles — a curious rational fraction that suggests a group-theoretic interpretation I haven't pinned down yet.

Would you like me to test specific structural predicates (pattern-avoidance classes, descent statistics, major index, genus), or to output the list of reached/unreached cycles for some n so you can inspect directly?

4/14/2026, 4:12:33 AM



please search for structural predicates

4/14/2026, 4:14:06 AM



Created file, ran command ▾

📄 Systematic structural predicate search

📄 Tool: bash_tool

✓ Done

Good — I found some real structural signals. Let me dig deeper on the strongest ones and also look at the (L, S) structure at leaves:

Created file, ran command ▾

📄 Focus on strongest predicates and L,S structure at leaves

📄 Tool: bash_tool

✓ Done

Strong findings. Here's the summary:

Necessary conditions (for $\text{root} = (\emptyset, \{1\}, \text{id})$)

Every reached n-cycle satisfies **both**:

- $\pi(n) \neq 1$ (in the cycle word, n is not immediately before 1)
- $\pi(\pi(1)) \neq n$ (in the cycle word, n is not at position 2)

These are necessary but not sufficient. The count of cycles satisfying both is

$(n-3) \cdot (n-2)!$, and reached is $(n-2)!$, so these eliminate a clean $(n-3)/(n-2)$ of cycles but leave a factor of $(n-3)$ more to rule out.

The 123-avoidance direction

- **Every reached cycle contains the pattern 132** in its cycle word $(1, a_2, \dots, a_n)$
- **Every cycle that avoids the pattern 123** (in the cycle word) is reached — but this is a very small set, not a characterization

Structure at n-cycle leaves

This is where it gets clean:

$L \cup S = [n]$ at every n -cycle leaf — so each reached n -cycle comes with a canonical partition of $[n]$ into two pieces L, S (with $1 \in S$ since the root has $S = \{1\}$ and S only grows).

Number of distinct S sets at n -cycle leaves = $2^{(n-3)}$:

n	distinct S -sets	$2^{(n-3)}$
5	4	4
6	8	8
7	16	16

So the reached cycles decompose along $2^{(n-3)}$ (L, S) -partitions. Moreover $1 \in S$ and $n \in L$ always (the extreme cycle $(1, n, n-1, \dots, 2)$ has $S = \{1\}, L = \{2, \dots, n\}$).

The single "unimodal" case

Exactly one reached cycle is unimodal from position 1: the strictly decreasing cycle $(1, n, n-1, \dots, 2)$. That's the leaf with $S = \{1\}, L = \{2, \dots, n\}$ — the path that only ever applies Rule A with $x = 1$, extending L one element at a time.

Bottom line

I couldn't find a single clean " π is reached iff X " characterization in terms of standard permutation statistics (descents, excedances, patterns, etc.). The algorithm seems to carve out a specific $(n-2)!$ -sized subset via a *process* that looks inherently non-local — each step's legality depends on the current (L, S, π) jointly. The counts strongly suggest an iterative/recursive description rather than a static one. If you want, I can try to find a **recursive** characterization: something like "the reached n -cycles are obtained from reached $(n-1)$ -cycles by inserting n in one of two specific ways."

4/14/2026, 4:16:55 AM



Are there two distinct cycles with the same S set?

4/14/2026, 4:23:57 AM



Yes, lots. Since $L \cup S = [n]$ at n -cycle leaves, knowing S also determines L , so the (L, S) pair alone doesn't determine π . Examples from the earlier output:

$n = 6, S = \{1, 2, 3\}$ — 8 distinct reached n -cycles:

Ran command ▾

Tool: bash_tool
Done

So for $n = 6$, the partition $S = \{1, 2, 3\}, L = \{4, 5, 6\}$ is shared by 8 different reached n -cycles. The multiplicities overall:

n	multiplicity distribution (S -set size $\rightarrow$ # of cycles)
5	1, 3, 1, 1
6	1, 7, 8, 1, 2, 3
7	1, 15, 37, 18, 1, 5, ... (across 16 S -sets)

Notably it's always the two extremes — $S = \{1\}$ and $S = \{1, 2, \dots, n-1\}$ — that correspond to exactly one cycle each (the strictly-decreasing $(1, n, n-1, \dots, 2)$ and the strictly-ascending $(1, 2, \dots, n)$ respectively). The "middle" partitions fan out into many cycles each.

4/14/2026, 4:24:19 AM



How many cycles are there per S ?

4/14/2026, 4:25:31 AM



Ran command ▾

Tool: bash_tool
Done

Very interesting structure emerges. Let me tally by $|S|$:

Ran command ▾

Tool: bash_tool
Done

Hmm wait — for $n=9$ the sum is 5035, not $5040 = 7!$. Let me double-check:

Ran command ▾

Tool: bash_tool
Done

Important correction — my $(n-2)!$ hypothesis breaks at $n=9$:

n	reached n-cycles	$(n-2)!$	diff
3..8	matches	matches	0
9	5035	5040	-5
10	40186	40320	-134

So the pattern held for small n and then fails. Let me look at the Eulerian-vs-actual row for $n=9$:

Ran command ▾

Tool: bash_tool
Done

Really striking result! Let me consolidate:

Counts by $|S|$ match Eulerian numbers — but only for $n \leq 8$

The Eulerian triangle $E(m, k)$ (permutations of $[m]$ with k ascents) is:

m=3:	1	4	1			
m=4:	1	11	11	1		
m=5:	1	26	66	26	1	
m=6:	1	57	302	302	57	1

And the $|S|$ -size distribution of reached n -cycles for $n = 5, \dots, 8$ matches row $m = n-2$ **exactly**. So for $n \leq 8$:

$$\#\{\text{reached } n\text{-cycles with } |S| = k\} = E(n-2, k-1)$$

totaling $(n-2)!$.

The pattern breaks at $n = 9$:

n	actual row	Eulerian row $m=n-2$	difference
9	1, 120, 1191, 2411 , 1191, 120, 1	1, 120, 1191, 2416 , 1191, 120, 1	middle term -5
10	1, 247, 4293, 15554 , 15550 , 4293, 247, 1	1, 247, 4293, 15619 , 15619 , 4293, 247, 1	middle terms -65, -69

Notice also that the symmetry $\text{count}(|S|=k) = \text{count}(|S|=n-k)$ that held for $n \leq 9$ **breaks** at $n = 10$ (15554 vs 15550).

Per-individual-S counts

For $S = \{1, k\}$ (exactly two elements): **count** = $2^{n-k-1} - 1$. This gives **1, 3, 7, 15, 31, ...** as k decreases and is exact across all n .

The extremes $S = \{1\}$ and $S = \{1, 2, \dots, n-1\}$ always give exactly 1 cycle each:

- $S = \{1\}$: the descending cycle $(1, n, n-1, \dots, 2)$
- $S = \{1, \dots, n-1\}$: the ascending cycle $(1, 2, 3, \dots, n)$

Takeaway

Up through $n = 8$ the structure looks almost too clean — Eulerian-by- $|S|$, $(n-2)!$ total, left-right symmetry. At $n = 9$ the middle Eulerian term is "missing 5", suggesting some obstruction kicks in when n is large enough that certain (L, S, π) paths that "should" be reachable get blocked by interactions between distant

elements. The deficit sequence `0, 0, 0, 0, 0, 0, 5, 134, ...` would be interesting to compute further. Want me to push to `n = 11, 12` and also break the deficit down by `|S|` to see which middle terms are losing mass?

4/14/2026, 4:27:12 AM



no. I want to prove that no two leaves reach the same cycle, can you help prove this result, or find a counter-example, and provide any needed relevant topics/background that would help work on a problem like this

4/14/2026, 4:29:03 AM



Let me first confirm the conjecture computationally across a wider range, checking *all* leaves (not just n-cycles):

Ran command ▾

📄 Tool: bash_tool

✓ Done

Very strong result — in fact **no two nodes anywhere in the tree share the same π** , not just no two leaves. Let me check whether there's a clean invariant that lets us reconstruct (L, S) from π alone:

Ran command ▾

📄 Tool: bash_tool

✓ Done

Excellent — this gives a clean proof. The conjecture is true and follows from a structural invariant.

The invariant

Claim. At every node (L, S, π) in the BFS tree rooted at $(\emptyset, \{1\}, \text{id})$:

$$L = \{i \in [n] : \pi(i) < i\} \quad (\text{anti-excedances})$$

$$S = \{1\} \cup \{i \in [n] : \pi(i) > i\} \quad (\text{excedances} \cup \{1\})$$

This immediately implies **π determines (L, S)** , so distinct nodes have distinct π — in particular no two leaves share a cycle. ■

Proof sketch (induction on depth)

Base. Root $(\emptyset, \{1\}, \text{id})$: $\pi = \text{id}$ has no excedances or anti-excedances, so the formulas give $L = \emptyset, S = \{1\}$ ✓.

Step. Suppose the invariant holds at (L, S, π) . Let $\pi' = \pi \circ \tau_{\{x,y\}}$, so $\pi'(x) = \pi(y)$, $\pi'(y) = \pi(x)$, unchanged elsewhere.

Rule A ($x \in S, y \notin L \cup S, \pi(y) = y, \pi(x) < y, x < y$; child adds y to L):

- At position x : $\pi'(x) = \pi(y) = y > x$, so x is an excedance of π' — stays in S . ✓
- At position y : $\pi'(y) = \pi(x) < y$, so y becomes an anti-excedance of π' — joins L . ✓
- All other positions: unchanged. So $L' = L \cup \{y\}$, $S' = S$ as required.

Rule B (symmetric): at x , $\pi'(x) = y < x$, so x stays in L ; at y , $\pi'(y) = \pi(x) > y$, so y joins excedances — joins S . ✓

The two rules are **literally the two ways to create a new excedance-anti-excedance pair by swapping a fixed point with an existing S- or L-element**. That's really all that's going on. ■

Verified empirically for $n \leq 8$, and the inductive argument extends to all n .

What "no two leaves share a cycle" gets you

- Each reachable (L, S, π) is uniquely labeled by π , so the tree injects into S_n .
- The image is $\{\pi : \pi \text{ is reachable}\}$, with L, S automatically recovered as the anti-excedance / excedance sets.
- Corollary: a π is a *leaf* iff it has no fixed point y satisfying Rule A or Rule B conditions — a condition entirely on π .

Relevant background / reading

If you want to situate this in the literature:

- Excedance / descent statistics.** The pair $(\# \text{excedances}, \# \text{anti-excedances})$ of a permutation is classical. Number of permutations of $[n]$ with k excedances = Eulerian number $E(n, k)$. Standard refs: Stanley, *Enumerative Combinatorics I*, §1.4; Bóna, *Combinatorics of Permutations*, Ch. 1. The fact that your $|S|$ -distribution matched Eulerian numbers up to $n \leq 8$ is no coincidence — it's

because at a leaf, $L \cup S = [n]$ (no fixed points = n -cycle or similar), so $|S| - 1$ is literally the number of excedances. The deviation at $n \geq 9$ then measures how many "Eulerian" excedance patterns fail to be constructible by the rules.

2. **Transition systems with invariants.** The technique of proving injectivity of a search tree by finding a function $f: \text{nodes} \rightarrow \text{labels}$ preserved by the transitions is standard in rewriting theory / term rewriting (look up "canonical forms" or "normal forms"). Baader & Nipkow, *Term Rewriting and All That*, is a standard reference.
3. **Permutation trees and reduced words.** What you have looks like a reduced-word / parking-function style construction where a permutation is built up by successive transpositions with constraints. Compare:
 - **Reduced words in S_n :** sequences of adjacent transpositions multiplying to π ; here you have non-adjacent transpositions involving fixed points.
 - **Stanley's parking functions** and their bijections with labeled trees — your $(n-2)!$ count on $n \leq 8$ and the Eulerian distribution might suggest a connection.
 - **Bubble-sort / insertion-sort trees** — building a permutation one excedance at a time.
4. **Genocchi / median Genocchi numbers.** These count permutations where excedance constraints are tightened — possibly relevant for explaining the $n \geq 9$ deviation, where some excedance patterns are not reachable.
5. **Sage / OEIS workflow.** To track down unfamiliar integer sequences, plug the deficit sequence $0, 0, 0, 0, 0, 0, 5, 134, \dots$ and the total reached sequence $1, 2, 6, 24, 120, 720, 5035, 40186$ into OEIS. If the deficit matches a known sequence you immediately get literature pointers.

Suggested next angles

- **Characterize the image:** which $\pi \in S_n$ are reachable? The invariant tells you the (L, S) structure is determined; what remains is describing which permutations arise. A natural guess: π is reachable iff some *recursive* condition holds (e.g., removing the largest element's cycle-predecessor leaves a reachable permutation on $[n-1]$). This would give a bijection with some tree structure.

- **Explain the $n \geq 9$ Eulerian deviation:** find the first π that has the right excedance set but isn't reachable, and see what goes wrong. For $n = 9$ there are exactly 5 "missing" permutations in the middle Eulerian class — small enough to eyeball.

Want me to dig into either?

4/14/2026, 4:30:50 AM



what are the number of leaves for n -trees for roots $(\emptyset, \{i\}, id)$ for $i = 1, \dots, n$

4/14/2026, 4:51:03 AM



Ran command ▾

Tool: bash_tool

✓ Done

Here are the leaf counts for root $(\emptyset, \{i\}, id)$, with i varying across columns:

n	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$	$i=6$	$i=7$	$i=8$	$i=9$	sum
2	1	1								2
3	2	1	1							4
4	4	3	2	1						10
5	10	11	7	4	1					33
6	33	42	35	24	10	1				145
7	145	195	198	146	90	33	1			808
8	808	1091	1197	1016	745	414	145	1		5417
9	5417	7219	8200	7736	6172	4293	2302	808	1	42148

Three clean patterns

1. Last column is always 1. Root $(\emptyset, \{n\}, id)$: Rule A needs $y > x = n$ (impossible) and Rule B needs $x \in L = \emptyset$. So the root is already a leaf.

2. Second-to-last column equals the $n-1$ first column. $leaves(n, n-1) = leaves(n-1, 1)$ — from $(\emptyset, \{n-1\}, id)$ the only first move is Rule A with $x = n-1, y = n$, and the resulting subtree is isomorphic to the $n-1$ problem.

3. First column equals the previous row's sum. $leaves(n, 1) = \sum_i leaves(n-1, i)$ — this is the relation that lets you generate the first column of each row from the

previous row:

- row sums: `2, 4, 10, 33, 145, 808, 5417, 42148, ...`
- these are exactly the first-column values shifted by one `n`.

So actually the whole table's top-row sequence `1, 2, 4, 10, 33, 145, 808, 5417, 42148, ...` is self-referential: each entry is the sum of the row indexed by the previous `n`.

The row sum

The row sum `T(n) =  $\sum_i \text{leaves}(n, i)$` satisfies `T(n+1) = leaves(n+1, 1)`, but we need another relation to pin down the whole row. The row sums `2, 4, 10, 33, 145, 808, 5417, 42148` don't match anything I recognize at a glance — worth checking OEIS.

Symmetry check

By the order-reversing symmetry from earlier, `leaves(n, i)` with root `( $\emptyset$ , {i}, id)` should equal `leaves(n, n+1-i)` with root `({n+1-i},  $\emptyset$ , id)`. That is, the table above is the "mirror" of the corresponding B-root table. Want me to check whether rows have any internal symmetry, or compute further columns (e.g., 2-element `s`-roots) for more structure?

4/14/2026, 4:51:59 AM



do the total sums equal the total number of n -cycles

4/14/2026, 4:55:29 AM



Let me check — the row sums are all leaves, but you might mean just n -cycle leaves:

Ran command ▾

Tool: bash_tool

Done

Close, but not exact. Here's what the data shows:

n	total n -cycle leaves (all i)	$(n-1)!$	distinct n -cycles reached
2	1	1	1
3	2	2	2
4	6	6	5
5	24	24	18
6	120	120	82
7	720	720	452
8	5035	5040	2935
9	40186	40320	21908

Two observations:

1. The total (counted with multiplicity across roots) equals $(n-1)!$ for $n \leq 7$, then breaks. So for small n there's a "conservation law": every n -cycle is covered with total multiplicity exactly $(n-1)!/(\text{number of distinct reached})$, but with multiplicities differing between cycles.

2. The **distinct** n -cycles reached is always strictly less than $(n-1)!$ for $n \geq 4$: 5, 18, 82, 452, 2935, 21908. So there exist n -cycles in S_n that are **not reachable as a leaf from any root** $(\emptyset, \{i\}, id)$ — even as we vary i .

So the union of all these trees (over $i = 1, \dots, n$) does **not** cover all n -cycles. The invariant from before — that L is the anti-excedances and $S \supseteq \{1\}$ is (something like) the excedances — now needs modification when the root is $(\emptyset, \{i\}, \text{id})$: repeating the inductive argument, you get

$$L = \{j : \pi(j) < j\}, \quad S = \{i\} \cup \{j : \pi(j) > j\}.$$

So each root $(\emptyset, \{i\}, \text{id})$ reaches a set of permutations whose (L, S) structure is pinned down by the excedances *plus* the distinguished element i (which must either be a fixed point of π , or be consistent with $\pi(i) > i$). Adding roots together doesn't necessarily cover all n -cycles because the reachability constraint is more than just "the excedance structure is right."

Would you like me to identify the first "missing" n -cycle — the smallest n -cycle of S_n not reached by any root $(\emptyset, \{i\}, \text{id})$ — and look for what structural obstruction it has?

4/14/2026, 4:56:13 AM



yes find the first missing n -cycle

4/14/2026, 4:58:22 AM



Ran command ▾

Tool: bash_tool

Done

The first missing n -cycle is for $n = 4$: the cycle $(1, 3, 4, 2)$ in cycle notation, i.e. $\pi = [3, 1, 4, 2]$.

Why this cycle is unreachable

Let's trace the obstruction using the invariant we proved:

Target $\pi = [3, 1, 4, 2]$:

- Excedances: $\pi(1) = 3 > 1$, $\pi(3) = 4 > 3$. So excedance set = $\{1, 3\}$.
- Anti-excedances: $\pi(2) = 1 < 2$, $\pi(4) = 2 < 4$. So L would be $\{2, 4\}$.

Under the modified invariant $S = \{i\} \cup \{\text{excedances}\}$, the only valid roots are $i = 1$ or $i = 3$. (For $i = 2$ or $i = 4$, the required S would overlap L .)

From root $(\emptyset, \{1\}, \text{id})$: to end with $\pi(1) = 3$, the transposition $\tau_{\{1,3\}}$ must occur at some Rule A step (since only Rule A involves $x = 1$). But immediately after that step, the rule adds 3 to L . **Once 3 is in L, it stays in L forever** (neither rule removes elements or moves them $L \rightarrow S$). So $3 \in L$ at the leaf — contradicting our target where $3 \in S$.

From root $(\emptyset, \{3\}, \text{id})$: the only first move is Rule A with $x = 3, y = 4$ (since y must be fixed and > 3). This gives $\pi = [1, 2, 4, 3]$, $L = \{4\}$, $S = \{3\}$. From here, to eventually get $\pi(1) = 3$ we need to swap 1 with 3 — but $3 \in S$ and 1 is a fixed point, and Rule A with $x = 3, y = 1$ requires $1 > 3$, which fails. So 1 can only enter via Rule B with $x = 4, y = 1$ (which adds 1 to S), producing $\pi = [2, 3, 4, 1]$ — the ascending cycle, dead-end leaf. The target is not reachable.

The general obstruction

Once an element j gets absorbed into L by Rule A (as y), or into S by Rule B (as y), it **cannot change bucket**. So the algorithm must build up L and S in an order compatible with the final excedance pattern. The cycle $(1, 3, 4, 2)$ has the "crossing" structure: from root $(\emptyset, \{1\})$, the first swap involving 3 commits 3 to L , but the target needs $3 \in S$.

Concretely, the reachable n -cycles from root $(\emptyset, \{i\}, id)$ are those π where there's an ordering of L -additions and S -additions (consistent with the rule conditions) that ends at the right excedance pattern. This is a **dynamical reachability** question — not purely a static combinatorial one — which is why no clean local invariant (descents, patterns, etc.) captures it.

Next steps you might explore

1. **Bijection with labeled trees / parking-function-like objects.** The $(n-2)!$ count for $n \leq 8$ and the Eulerian-like distribution suggest the reachable n -cycles from $(\emptyset, \{1\}, id)$ might be in bijection with a known combinatorial family with a natural "insertion order" — e.g., increasing binary trees on $n-1$ nodes, or parking functions with some constraint.
2. **Characterize via "excedance insertion sequences."** For each reachable π , there's a specific order in which elements entered L vs S . If two different orderings can produce the same π , then the algorithm "branches" onto the same endpoint — but our invariant proof shows this doesn't happen within one tree. Across different roots, however, the same π can be reached multiple times (as you saw earlier).
3. **OEIS hunt.** Key sequences worth searching:
 - Total reachable n -cycles summed across roots $(\emptyset, \{i\})$: $1, 2, 6, 24, 120, 720, 5035, 40186$ — matches $(n-1)!$ up to $n=7$, then deviates.
 - Distinct reachable n -cycles: $1, 2, 5, 18, 82, 452, 2935, 21908$ — doesn't ring a bell.
 - Missing n -cycles: $0, 0, 1, 6, 38, 268, 2105, 18412$.

Any hit on OEIS would give strong pointers.

4/14/2026, 4:59:43 AM



update the algorithm so that we have a third way to add a child, pick x, y not in $L \cup S$ such that $x < y$ and create the child $(S \cup \{x\}, L \cup \{y\}, \pi \circ \tau_{xy})$, and start from the root $(\emptyset, \emptyset, id)$, does the invariant still hold?

4/14/2026, 5:14:09 AM



Created file, ran command ▾

- ⊞ Updated BFS with Rule C, check invariant
- |
- 📄 Tool: bash_tool
- |
- 👌 Done

Yes — the invariant **still holds, and in a cleaner form**. With Rule C added and starting from $(\emptyset, \emptyset, id)$:

$$L = \{i : \pi(i) < i\} \quad (\text{anti-excedances})$$

$$S = \{i : \pi(i) > i\} \quad (\text{excedances})$$

No more distinguished $\{1\}$ or $\{i\}$ — just the clean excedance / anti-excedance decomposition. Verified empirically up to $n = 7$: at every node, the invariant matches, and distinct nodes have distinct π .

Why Rule C preserves the invariant

By the invariant at the parent (L, S, π) :

- $x \notin L \cup S$ means $\pi(x) \geq x$ (not anti-excedance) and $\pi(x) \leq x$ (not excedance), so $\pi(x) = x$. Same for y .

(So even though Rule C doesn't *explicitly* require $\pi(x) = \pi(y) = y$, the invariant forces both to be fixed points.)

After swapping: $\pi'(x) = \pi(y) = y > x$ (excedance $\rightarrow$ joins S), $\pi'(y) = \pi(x) = x < y$ (anti-excedance $\rightarrow$ joins L). All other positions unchanged. So $L' = L \cup \{y\}$, $S' = S \cup \{x\}$ as required. ✓

New tree statistics

n	total nodes	leaves
2	2	1
3	6	3
4	23	10
5	109	44
6	616	236
7	4044	1493

Interpretation

With the addition of Rule C, the system now has *three* local transitions, each corresponding to one of the three ways to create a new (excedance, anti-excedance) pair by swapping a fixed point with something — either a fixed point (Rule C), an existing excedance (Rule A), or an existing anti-excedance (Rule B). The invariant tells you the state `(L, S)` is redundant: it's always just `(anti-excedances of  $\pi$ , excedances of  $\pi$ )`. So this system is really just a rewriting rule on **permutations** — with `L, S` as convenient bookkeeping.

Want me to check whether the image of the tree (the set of reachable permutations) is now all of `Sn`, or still a strict subset? My guess: still a strict subset, because Rule C only swaps **two fixed points**, which is a very constrained operation — you can never "edit" an already-non-fixed portion of π .

4/14/2026, 5:15:08 AM



now check if all the leaves correspond to n -cycles

4/14/2026, 5:16:07 AM



Ran command ▾

Tool: bash_tool

Done

Not all leaves are n -cycles. Full breakdown:

n	leaves	n -cycles	cycle types seen
3	3	2	(3), (2,1)
4	10	5	(4), (3,1), (2,2)
5	44	18	(5), (4,1), (3,2), (2,2,1)
6	236	82	(6), (5,1), (4,2), (3,3), (3,2,1), (2,2,2)
7	1493	452	(7), (6,1), (5,2), (4,3), (4,2,1), (3,3,1), (3,2,2), (2,2,2,1)

Two structural observations:

1. A leaf has at most one fixed point. This follows immediately from the rules: Rule C needs ≥ 2 elements outside LUS , and under the invariant "outside LUS " = "fixed points of π ". So with ≥ 2 fixed points, Rule C always fires $\rightarrow$ not a leaf.

2. Leaves come in two flavors:

- Derangements** (no fixed points) — no rule can fire because A/B/C all need at least one fixed point.
- "Stuck" permutations with exactly one fixed point** — the one fixed point y fails the inequalities of Rules A and B for every $x \in S$ and $x \in L$.

So n -cycles are just one of many possible leaf cycle types. Let me also check whether Rule C unlocked the previously-missing n -cycle $(1, 3, 4, 2)$:

Ran command ▾

Tool: bash_tool

Done

Interestingly, Rule C does **not** expand the reachable set — the same n-cycles remain unreachable:

n	Rule C: n-cycles reached	Before (multi-root): n-cycles reached
4	5/6	5/6
5	18/24	18/24
6	82/120	82/120
7	452/720	452/720

Exactly the same counts. And the first missing n-cycle is still $(1, 3, 4, 2)$. The obstruction is the same: once an element enters L (is committed to being an anti-excedance), it stays there, and there's no path to produce $\pi = [3, 1, 4, 2]$ that avoids prematurely committing 3 or 4 to the wrong bucket.

Summary of what Rule C buys you

Rule C is equivalent to the operation "spawn a new excedance/anti-excedance pair from two fixed points $x < y$." This is exactly the operation that lets you simulate picking any starting root $(\emptyset, \{i\}, id)$ at any point in the tree, not just at time zero. So:

- Structurally:** the single-root-with-Rule-C tree is the "disjoint union" of all the single- $\{i\}$ -root trees, glued via Rule C transitions. It covers the same set of reachable permutations.
- Invariant:** cleaner — (L, S) is exactly $(\text{anti-excedances}, \text{excedances})$, no distinguished element.
- Reachability:** identical set of n-cycles reached.

Leaves = derangements $\cup$ (single-fixed-point "stuck" perms)

As noted, a leaf has at most one fixed point (since Rule C always fires with ≥ 2 fixed points). The leaves fall into two families:

- **Derangements** (0 fixed points): cycle types with all parts ≥ 2 . For $n = 7$ these are $(7), (5,2), (4,3), (3,2,2), (2,2,2,1)$ — wait that last has a 1; actually derangements have all parts ≥ 2 , so $(7), (5,2), (4,3), (3,2,2)$. Let me re-examine.

Looking again at $n = 7$ leaf cycle types: $(7), (6,1), (5,2), (4,3), (4,2,1), (3,3,1), (3,2,2), (2,2,2,1)$ — the ones with a single 1 are "stuck with one fixed point", rest are derangements.

What's the characterization of reachable permutations?

Since the set of reachable permutations equals the set reachable from some $(\emptyset, \{i\}, \text{id})$ root (Rule C = starting fresh from any pair), and we know the $\{1\}$ root gives a clean excedance structure, the reachable permutations form a union of "excedance-compatible" orbits. The fact that $(1,3,4,2)$ is unreachable tells us there's a *sequential* obstruction that no purely local invariant captures. If you want to nail this down, I'd suggest looking at **Lehmer code / inversion sequence** representations of the reachable permutations — the rules translate into insertion operations on these encodings, which may reveal a cleaner pattern-avoidance characterization.

4/14/2026, 5:18:10 AM



update rule C so that instead of picking x, y not in $L \cup S$, we just pick x not in L and y not in S

4/14/2026, 5:21:55 AM



Created file, ran command ▾

- 📄 BFS with updated Rule C
- 📁 Tool: bash_tool
- 👉 Done

The update **breaks everything**:

n	nodes	leaves	invariant holds?	distinct π ?	n-cycles reached
2	3	0	✗	✗	0
3	19	0	✗	✗	0
4	157	0	✗	✗	0
5	1581	0	✗	✗	0
6	18991	0	✗	✗	0
7	265903	0	✗	✗	0

Three things went wrong:

1. The invariant fails immediately

First violation at $n = 2$: starting from $(\emptyset, \emptyset, [1, 2])$:

- Apply Rule C with $x = 1, y = 2$: child $(L=\{2\}, S=\{1\}, \pi=[2, 1])$ ✓ invariant fine so far.
- Apply Rule C again with $x = 1, y = 2$ (now $1 \notin L=\{2\}$ and $2 \notin S=\{1\}$, so still allowed): swap $1, 2$ again $\rightarrow \pi$ becomes identity $[1, 2]$ again. L and S are unions so they stay $\{2\}$ and $\{1\}$.

Result: state $(L=\{2\}, S=\{1\}, \pi=id)$. But $\pi = id$ has no excedances or anti-excedances, so the invariant predicts $L = S = \emptyset$. **Contradiction** — the rule lets you "un-swap" and the L/S bookkeeping doesn't undo.

2. No leaves

The same pair (x, y) can be used repeatedly — L and S accumulate but π can be toggled back and forth. In fact, for any state where not everything is in L or S, you can always find some $x \notin L, y \notin S$ with $x < y$ (e.g., pick $x = \min([n] \setminus L)$ and $y = \max([n] \setminus S)$; typically $x < y$). So Rule C almost never disables itself, and there are **no leaves** at all — every node has a child.

3. Distinct π 's per node is lost

Because L and S keep growing monotonically but π can be toggled, many different (L, S) pairs share the same π . The invariant $\pi \rightarrow (L, S)$ reconstruction is dead.

Why the original version worked

The original Rules A/B/C all *required* fixed points — either the swap was between a bucket element and a fixed point (A/B) or between two fixed points (C). Swapping a fixed point with anything always changes π in a way that **can't be undone by the same operation**, because after the swap neither element is a fixed point anymore, so that specific rule-instance can't be applied again. This is what prevents revisits and makes the structural invariant work.

The loosened Rule C removes this safety. Once the swap has been made, the "inverse swap" (which would be the same (x, y) applied again) is no longer prevented, because the rule doesn't check fixed-point status. So you get oscillations and an infinite-regress-like behavior.

If you want a loosened Rule C that *does* preserve the invariant, you'd need to add a fixed-point requirement on at least one of x, y — which brings you back toward Rules A/B or the original Rule C. If the goal is something more permissive but still well-behaved, one option would be: $x \notin L, y \notin S, x < y$, **and additionally** $\pi(x) < y$ (so that after the swap y genuinely becomes an anti-excedance). Want me to test that variant?

4/14/2026, 5:23:44 AM



replace all the rules A,B,C with a single rule, pick x not in L , y not in S such that $x \leq \pi(x) < \pi(y) \leq y$ and create the new child $(L \cup \{y\}, S \cup \{x\}, \pi \circ \tau_{x,y})$

4/14/2026, 5:34:12 AM



Created file, ran command ▾

📄 Unified single-rule BFS

📁 Tool: bash_tool

✅ Done

Beautiful — this unified rule gives a much cleaner structure:

n	nodes	leaves	invariant	distinct π
2	2	1	✓	✓
3	6	3	✓	✓
4	24	9	✓	✓
5	120	35	✓	✓
6	720	161	✓	✓
7	5040	859	✓	✓
8	40320	5209	✓	✓

Key facts

1. Total nodes = $n!$ — so the tree's image is **exactly all of S_n** . Every permutation appears exactly once as a node, with (L, S) determined by the invariant. This is the cleanest possible statement.

2. Invariant preserved. The rule $x \leq \pi(x) < \pi(y) \leq y$ is crafted precisely so that after the swap, $\pi'(x) = \pi(y) > \pi(x) \geq x$ (x becomes excedance $\rightarrow$ joins S) and $\pi'(y) = \pi(x) < \pi(y) \leq y$ (y becomes anti-excedance $\rightarrow$ joins L). Also $x \notin L$ and $y \notin S$ ensure the "joins" are consistent.

3. Distinct π per node — same proof as before.

Proof sketch that all of S_n is reachable

Since total nodes = $n!$ and π 's are distinct across nodes, the BFS image equals S_n . Alternatively, you can show reachability directly: for any $\pi \in S_n$, you can build it by processing pairs $(i, \pi(i))$ with $\pi(i) > i$ in decreasing order of i , each time picking an appropriate (x, y) . This is worth writing out carefully if you want a formal proof.

Leaf structure

Leaves are permutations where no rule fires: for every $x \notin L$ (i.e., $\pi(x) \geq x$) and every $y \notin S$ (i.e., $\pi(y) \leq y$), the inequality $\pi(x) < \pi(y)$ fails — i.e., $\pi(x) \geq \pi(y)$. Equivalently, the leaves are permutations π such that for every **weak excedance position** x and every **weak anti-excedance position** y with $\pi(x) < \pi(y)$ (the "displacement" condition), the pair fails. In particular the cycle-type distribution is:

- $n=4$: $5 \cdot (4), 2 \cdot (3,1), 2 \cdot (2,2)$
- $n=5$: $16 \cdot (5), 5 \cdot (4,1), 12 \cdot (3,2), 2 \cdot (2,2,1)$
- $n=6$: $63 \cdot (6), 16 \cdot (5,1), 43 \cdot (4,2), 21 \cdot (3,3), 12 \cdot (3,2,1), 6 \cdot (2,2,2)$
- $n=7$: $294 \cdot (7), 63 \cdot (6,1), 188 \cdot (5,2), 172 \cdot (4,3), 43 \cdot (4,2,1), 21 \cdot (3,3,1), 72 \cdot (3,2,2), 6 \cdot (2,2,2,1)$

The leaf-count sequence $1, 3, 9, 35, 161, 859, 5209$ is worth checking on OEIS — it may be a known sequence (looks like an Euler-ish number).

n-cycle coverage is still incomplete

Even though *all* permutations are in the tree, **not all n-cycles are leaves** — some n-cycles have children and aren't stuck:

n	n-cycle leaves	total n-cycles
4	5	6
5	16	24
6	63	120

n	n-cycle leaves	total n-cycles
7	294	720

The missing n-cycle for $n = 4$ is still $(1, 3, 4, 2)$: its excedance structure is $L = \{2, 4\}$, $S = \{1, 3\}$, and checking the leaf condition, no rule fires there — so it IS a leaf. Wait, that means it's counted as a leaf. Let me re-examine — actually the coverage count $5 < 6$ suggests one n-cycle is *not* a leaf (has a child). Let me check quickly:

Ran command ▾

Tool: bash_tool

Done

Confirmed — some n-cycles are intermediate nodes, not leaves. E.g., for $n = 4$: $\pi = [2, 4, 1, 3]$ (the 4-cycle $(1, 2, 4, 3)$) has a child $\pi = [3, 4, 1, 2]$ (which has cycle type $(2, 2)$). So the rule can transition from an n-cycle to a permutation with a 1-dim smaller cycle structure — surprising because $L \cup S = [n]$ already at the parent.

Actually, wait — the rule fires here because $x = 1 \notin L = \{3, 4\}$ with $\pi(x) = 2$, and $y = 3 \notin S = \{1, 2\}$ with $\pi(y) = 1$... but $\pi(x) = 2 < \pi(y) = 1$ fails. Let me recheck:

Ran command ▾

Tool: bash_tool

Done

Got it — $x=1, y=4$ (I was picking the wrong y above). The rule fires and turns a 4-cycle into a product of two 2-cycles. So the transition from n-cycle to $(2, 2)$ is real — a transposition applied to an n-cycle can split it into two cycles when the transposed elements are in the same cycle.

Clean summary

The unified rule gives a **BFS exploration of all of S_n** , with:

1. **Exactly $n!$ nodes** — every permutation appears exactly once.
2. **The invariant $L = \text{anti-excedances}, S = \text{excedances}$** holds throughout (proof as before — the rule's inequality $x \leq \pi(x) < \pi(y) \leq y$ is designed to flip x into excedance and y into anti-excedance).
3. **(L, S) is completely determined by π** , so the triple (L, S, π) is just π + bookkeeping.

4. **Distinct nodes have distinct π** $\implies$ BFS tree is a spanning tree of S_n under the transition relation.
5. **Leaves are permutations where no (x, y) satisfies the inequality** — a purely π -local condition.
6. The cycle-type distribution at leaves shows a rich combinatorial structure; leaf count sequence is $1, 3, 9, 35, 161, 859, 5209, \dots$.

The rule is essentially a **generator for all of S_n** starting from the identity by a specific family of transpositions. Each transposition $\tau_{\{x,y\}}$ is "legal" iff it creates a new excedance-anti-excedance pair (x, y) with the pre-swap values strictly increasing ($\pi(x) < \pi(y)$). It's a very natural construction — worth checking if it corresponds to a known "sorting network" or Bruhat-order-like structure on S_n .

Would you like me to:

- Check OEIS for the leaf sequence $1, 3, 9, 35, 161, 859, 5209$?
- Look at how the BFS depth profile (nodes per depth) relates to classical S_n statistics (like number of excedances or major index)?
- Characterize leaves as "permutations avoiding a specific local pattern" more formally?

4/14/2026, 6:05:22 AM



how do i prove the total number of nodes is $n!$

4/14/2026, 5:40:13 AM



Here's a proof sketch. The key pieces are already in hand — you have the invariant and distinct- π -per-node results, so injectivity is done. What remains is surjectivity, and the tool is a displacement-based induction.

Proof that $|tree| = n!$

Setup. We already have:

1. **Invariant** (proved): at every tree node, $L = \{i : \pi(i) < i\}$ and $S = \{i : \pi(i) > i\}$.
2. **Injectivity** (corollary): distinct nodes have distinct π , because (L, S) is determined by π .

So the map $(L, S, \pi) \mapsto \pi$ embeds the tree into S_n . It remains to show the image is all of S_n .

Displacement. Define

$$D(\pi) = \sum_{i=1}^n |\pi(i) - i|.$$

Note $D(\pi) \geq 0$ with equality iff $\pi = id$, and $D(\pi)$ is always even.

Step-change computation. Suppose the rule fires at π with (x, y) satisfying $x \leq \pi(x) < \pi(y) \leq y$ (both with $x \notin L$, $y \notin S$). Let $\pi' = \pi \circ \tau_{\{x,y\}}$.

Contribution of positions x, y to D :

- Before: $(\pi(x) - x) + (y - \pi(y))$, since $\pi(x) \geq x$ and $\pi(y) \leq y$.
- After: $\pi'(x) = \pi(y)$, $\pi'(y) = \pi(x)$. From $x \leq \pi(y)$ and $\pi(x) \leq y$, both signs are preserved: $(\pi(y) - x) + (y - \pi(x))$.

Change: $[(\pi(y) - x) + (y - \pi(x))] - [(\pi(x) - x) + (y - \pi(y))] = 2(\pi(y) - \pi(x)) > 0$.

So **each rule application strictly increases D by $2(\pi(y) - \pi(x)) \geq 2$** .

Equivalently, each **reverse** step strictly decreases D . This makes the proof an induction.

Surjectivity by induction on $D(\pi)$.

Base: $D(\pi) = 0$ means $\pi = \text{id}$, which is the root.

Induction: Assume every permutation with displacement $< D$ is reachable, and let π have displacement $D > 0$. We exhibit a valid predecessor π'' with $D(\pi'') < D$, then apply the rule at π'' to produce π .

Key lemma. For every $\pi \neq \text{id}$, there exist $x \in \text{exc}(\pi)$ and $y \in \text{antiexc}(\pi)$ with

$$x \leq \pi(y) < \pi(x) \leq y.$$

Given the lemma, set $\pi'' = \pi \circ \tau_{\{x, y\}}$. Direct check: $\pi''(x) = \pi(y)$, $\pi''(y) = \pi(x)$.

The displacement at x, y decreases by $2(\pi(x) - \pi(y)) \geq 2$; everywhere else it's unchanged. So $D(\pi'') < D(\pi)$.

By induction π'' is reachable, so its (L, S) matches $(\text{antiexc}(\pi''), \text{exc}(\pi''))$. From there, applying the rule with the same (x, y) gives π as a child — one needs to verify the preconditions at π'' :

- $x \notin L(\pi'')$: $\pi''(x) = \pi(y) \geq x$, so x is not an anti-excedance of π'' . ✓
- $y \notin S(\pi'')$: $\pi''(y) = \pi(x) \leq y$, so y is not an excedance of π'' . ✓
- $x \leq \pi''(x) < \pi''(y) \leq y$: this is $x \leq \pi(y) < \pi(x) \leq y$, our lemma. ✓

Proving the key lemma

Take $x^* = \min \text{exc}(\pi)$ and $y^* = \max \text{antiexc}(\pi)$. Claim: this pair works.

Inequalities to check (with π fixed):

- $\pi(x^*) \leq y^*$: Suppose for contradiction that $\pi(x^*) > y^*$. Then the value $\pi(x^*)$ is a position strictly above y^* ; but all positions $> y^*$ that are excedances are no issue, and positions $> y^*$ that are anti-excedances would contradict maximality of y^* . More carefully, let $v = \pi(x^*) > y^*$. If v is fixed, $\pi(v) = v$, but also $\pi(x^*) = v$ forces $x^* = v$, contradicting $x^* \in \text{exc}$ (means $x^* < \pi(x^*) = v$). If $v \in$

antiexc, then $v > y^*$ is a larger anti-excedance, contradicting maximality. If $v \in \text{exc}$, then $\pi(v) > v > y^*$, and iterating we get an infinite ascending chain — impossible in a finite cycle.

- $\pi(y^*) \geq x^*$: symmetric argument.
- $\pi(y^*) < \pi(x^*)$: $\pi(y^*) < y^*$ (since y^* is an anti-excedance) and $\pi(x^*) > x^*$ (since x^* is an excedance). Combined with what we just showed, $\pi(y^*) \leq y^*$ and $\pi(x^*) \geq x^* + 1$. We need to rule out equality/ordering — use a case analysis, or use that $x^* \leq \pi(y^*)$ and $\pi(x^*) \leq y^*$ force both values into $[x^*, y^*]$, and if they coincided you'd get contradictions with the exc/anti-exc structure.

(The full argument needs care — I'd recommend writing it out on paper or in Lean/Rocq; you can verify each step computationally to sanity-check.)

Cleaner alternative: use the tree structure directly

There's also a slicker route that sidesteps the lemma. You can directly construct a path $\text{id} \rightarrow \pi$ for any π as follows:

1. Let σ be a sequence of transpositions whose product is π , built by processing the cycles of π .
2. For each transposition in the sequence, check that it corresponds to a valid rule firing given the running excedance/anti-excedance state.

In fact, any permutation can be built by the greedy "assign the target value to position i for $i = 1, 2, \dots, n$ in order" procedure, and each such assignment swap is a valid rule application under a careful analysis.

What to verify

Empirically the computation showed $|\text{tree}| = n!$ for $n \leq 8$. The proof strategy is:

1. **Injectivity** — already proven via the invariant.
2. **Displacement monotonicity** — just calculated.
3. **Key Lemma** — the main work; $(x^*, y^*) = (\min \text{exc}, \max \text{antiexc})$ is a natural candidate and the empirical evidence supports it.

If you want, I can try to nail down the key lemma more rigorously (e.g., by checking all cases on a finite set of permutations and pointing out where the contradiction-based

argument tightens up) or sketch the "greedy assignment" alternative. Which would be more useful?

4/14/2026, 5:42:56 AM



write out a latex document doing the following

- defining the latest tree and BFS procedure
- stating and proving the invariant about the nodes of the tree
- proving that there is a node for every permutation by using the key lemma argument

4/14/2026, 5:53:40 AM



Let me first verify computationally which choices of witness (x, y) always work, then write the document.

Ran command ▾

Tool: bash_tool
Done

Let me first work out a clean proof of the key lemma. I found that a choice based on $y^* = \max B(\pi)$ works:

Ran command ▾

Tool: bash_tool
Done

The witness choice works — now let me write the LaTeX document.

Created file, ran 2 commands, edited file, presented file ▾

LaTeX document with definition, invariant, and surjectivity proof
Tool: bash_tool
Tool: str_replace
Tool: bash_tool
Presenting file(s)...
Done

The document has three parts:

§1. Definitions. The unified rule — pick (x, y) with $x \notin L$, $y \notin S$, $x \nmid \pi(x) < \pi(y) \nmid y$; child is $(L \cup \{y\}, S \cup \{x\}, \pi \circ \tau_{x,y})$ — and the BFS that

builds T_n from $(\emptyset, \emptyset, \text{id})$.

§2. The invariant ($L = \text{aex}(\pi)$, $S = \text{exc}(\pi)$ at every node). Induction on depth, tracking what happens at positions x and y after the swap. The rule's inequality is engineered exactly so x becomes an excedance and y becomes an anti-excedance of the child. Injectivity of π across nodes is an immediate corollary.

§3. Surjectivity. Induction on $D(\pi) = \sum_i |\pi(i) - i|$. The key lemma: for $\pi \neq \text{id}$, with $y^* = \max \text{aex}(\pi)$ and $v = \pi(y^*)$, there exists $x \leq v$ with $\pi(x) > v$, and any such x satisfies $\pi(x) \leq y^*$. Part (a) is a bijection argument on $[1, v]$; part (b) follows by iterating π from x and observing that once you exceed y^* you can only stay above it (no anti-excedances up there), which blows up the finite orbit.

Given the lemma, $\pi'' = \pi \circ \tau_{x,y^*}$ has $D(\pi'') - D(\pi) = 2(v - \pi(x)) < 0$, and the rule verifiably fires at π'' with (x, y^*) producing π — so induction on D closes.

I verified computationally that this specific witness choice works for all non-identity π up to $n = 8$.

 tree Pdf 

 tree Tex 

4/14/2026, 6:05:58 AM



you can induct on the length of the permutation (number of inversions), no need for using displacement. rewrite the proof and latex

4/14/2026, 6:16:09 AM



Ran 3 commands, created 2 files, presented file ▾

- Tool: bash_tool
- rewrite using inversion count
- Tool: bash_tool
- rewrite using inversion count
- Tool: bash_tool
- Presenting file(s)...
- Done

The induction on $\text{inv}(\pi)$ replaces the displacement argument. The structure of the proof is otherwise identical, with one addition and one substitution:

Substitution. Instead of directly computing $D(\pi'') - D(\pi) = 2(v - \pi(x)) < 0$, I invoke the standard transposition-inversion lemma:

$$\text{inv}(\pi \circ \tau_{a,b}) < \text{inv}(\pi) \iff \pi(a) > \pi(b) \quad (a < b).$$

Applied with $(a, b) = (x, y^*)$, we need $\pi(x) > \pi(y^*) = v$, which is exactly the defining property of x from the Key Lemma. So $\text{inv}(\pi'') < \text{inv}(\pi)$ is immediate.

Addition. I included a proof of the transposition-inversion lemma (Lemma 3) — pairs disjoint from $\{a, b\}$ contribute nothing, the pair (a, b) itself flips sign, and pairs meeting $\{a, b\}$ in one element contribute in pairs (even total), so the change has the same sign as the (a, b) contribution.

The induction on $\text{inv}(\pi)$ is arguably cleaner than on $D(\pi)$ because it ties directly to a well-known group-theoretic quantity (Coxeter length in S_n), so the "shorter predecessor" framing is standard.



tree



Pdf



tree
Tex



4/14/2026, 6:19:25 AM



recall the definition of good and increasing reps from another chat

4/14/2026, 2:30:07 PM



I don't have memory of other chats — each conversation starts fresh for me, and memory isn't enabled on your account. So I can't recall definitions from a previous session.

If you paste the definitions (or the relevant excerpt) here, I can pick up from there. Alternatively, if the definitions came up earlier in *this* conversation's transcript, let me know and I can search it — but I don't see "good reps" or "increasing reps" in the compaction summary at the top.

4/14/2026, 2:30:14 PM



A transposition σ_{ij} is a permutation that just swaps i and j . A collection of transpositions $C = \{\sigma_{ij}\}$ is good if it is such that for any σ_{ij} in C and $i < j$ there is no transposition σ_{ki} in C such that $k < i$ (because i was the smaller number in one of the transpositions it cannot be the larger number in any other transposition in the collection). Formalize this definition and explain it back and give some examples

4/14/2026, 2:30:34 PM



Formalization

Fix $n \geq 1$. A transposition on $[n]$ is specified by an unordered pair $\{i, j\} \subseteq [n]$ with $i \neq j$; write σ_{ij} for the permutation swapping them. For such a pair, let

$$\min(\sigma_{ij}) := \min(i, j), \quad \max(\sigma_{ij}) := \max(i, j).$$

Let C be a finite collection of distinct transpositions on $[n]$. Define

$$\text{Mins}(C) := \{\min(\sigma) : \sigma \in C\}, \quad \text{Maxs}(C) := \{\max(\sigma) : \sigma \in C\}.$$

Definition. C is **good** if $\text{Mins}(C) \cap \text{Maxs}(C) = \emptyset$.

Equivalently: for every $\sigma_{ij} \in C$ with $i < j$, there is no $\sigma_{ki} \in C$ with $k < i$.

Explanation

Think of each transposition as an edge on vertex set $[n]$, oriented from the smaller endpoint to the larger. An element $i \in [n]$ can play three roles across C :

- always the smaller endpoint (a "source"),
- always the larger endpoint (a "sink"),
- or both (a "conflict" — appears as min somewhere and as max elsewhere).

C is good iff no element is a conflict. The vertices $[n]$ split into sources, sinks, and isolated vertices, and every edge goes from a source to a sink. So the underlying

graph is bipartite with the bipartition ($\text{Mins}(C), \text{Maxs}(C)$) forced by the orientation.

Examples

Take $n = 5$.

- $C_1 = \{\sigma_{13}, \sigma_{14}, \sigma_{25}\}$. $\text{Mins} = \{1, 2\}$, $\text{Maxs} = \{3, 4, 5\}$. Disjoint — **good**.
- $C_2 = \{\sigma_{12}, \sigma_{23}\}$. $\text{Mins} = \{1, 2\}$, $\text{Maxs} = \{2, 3\}$. Element 2 is both — **not good**.
(It's the larger in σ_{12} and the smaller in σ_{23} .)
- $C_3 = \emptyset$ and any singleton $\{\sigma_{ij}\}$: vacuously/trivially **good**.
- $C_4 = \{\sigma_{14}, \sigma_{24}, \sigma_{34}\}$ (a "star" into 4). $\text{Mins} = \{1, 2, 3\}$, $\text{Maxs} = \{4\}$. Disjoint — **good**.
- $C_5 = \{\sigma_{12}, \sigma_{34}, \sigma_{25}\}$. $\text{Mins} = \{1, 3, 2\}$, $\text{Maxs} = \{2, 4, 5\}$. Element 2 conflicts — **not good**.
- $C_6 = \{\sigma_{13}, \sigma_{35}\}$. $\text{Mins} = \{1, 3\}$, $\text{Maxs} = \{3, 5\}$. Element 3 conflicts — **not good**.

A useful sanity check: if C is good, then any chain $i_1 < i_2 < \dots < i_k$ of elements cannot be "walked" by transpositions in C — i.e., you cannot have $\sigma_{i_1 i_2}, \sigma_{i_2 i_3} \in C$, since that would make i_2 both a max and a min. Good collections are thus "depth ≤ 1 " as oriented graphs.

Now — what's the next definition, "increasing reps"?

4/14/2026, 2:31:03 PM



An increasing representation of a permutation π is a composition of transpositions $\pi = \sigma_1 \circ \sigma_2 \circ \dots \circ \sigma_m$ such that the sequence of lengths $\ell_i = \ell(\pi_i)$ ($\sigma_1 \circ \dots \circ \sigma_i$) is a strictly increasing sequence. Formalize the definition of increasing representation.

4/14/2026, 2:31:31 PM



Formalization

Let $\ell : S_n \rightarrow \mathbb{Z}_{\geq 0}$ denote the **Coxeter length** (equivalently, the number of inversions):

$$\ell(\pi) := \text{inv}(\pi) = \#\{(i, j) : 1 \leq i < j \leq n, \pi(i) > \pi(j)\}.$$

Definition. Let $\pi \in S_n$. An **increasing representation** of π is a tuple $(\sigma_1, \sigma_2, \dots, \sigma_m)$ of transpositions in S_n (with $m \geq 0$) such that:

1. $\pi = \sigma_1 \circ \sigma_2 \circ \dots \circ \sigma_m$, and
2. setting $\pi_i := \sigma_1 \circ \sigma_2 \circ \dots \circ \sigma_i$ for $0 \leq i \leq m$ (so $\pi_0 = \text{id}$ and $\pi_m = \pi$), the sequence

$$\ell_0 = \ell(\pi_0), \ell_1 = \ell(\pi_1), \dots, \ell_m = \ell(\pi_m)$$

is **strictly increasing**: $\ell_0 < \ell_1 < \dots < \ell_m$.

Since $\ell_0 = 0$ and $\ell_m = \ell(\pi)$, strict increase forces $m \leq \ell(\pi)$.

A couple of quick remarks

Because each σ_i is a transposition, the general transposition-inversion lemma gives

$$\ell(\pi_i) - \ell(\pi_{i-1}) = \ell(\pi_{i-1} \circ \sigma_i) - \ell(\pi_{i-1}) \in \{\text{odd integers}\},$$

and in particular $\ell_i \neq \ell_{i-1}$. So the condition "strictly increasing" is equivalent to the

weaker-looking "each step does not decrease length": $\ell_i > \ell_{i-1}$ for all i .

The empty tuple ($m = 0$) is an increasing representation of id , and it is the only one (any nonempty product of transpositions equaling id must have some step that decreases length).

When $m = \ell(\pi)$, each step increases length by exactly 1, recovering the notion of a **reduced expression** of π in transpositions — but increasing representations are more general, allowing jumps of $+3, +5, \dots$

Ready for the next piece — presumably how good collections relate to increasing representations?

4/14/2026, 2:31:45 PM



Let me redefine "good". A collection of transpositions $C = \{\sigma_{ij}\}$ is good if the sets $S = \{i \mid \sigma_{ij} \in C, i < j\}$ and $L = \{j \mid \sigma_{ij} \in C, i < j\}$ are disjoint

4/14/2026, 2:36:30 PM



Formalization

Let C be a finite set of distinct transpositions on $[n]$. For each $\sigma \in C$, write $\sigma = \sigma_{ij}$ with $i < j$; call i the **small endpoint** and j the **large endpoint**. Define

$$S(C) := \{i : \sigma_{ij} \in C, i < j\}, \quad L(C) := \{j : \sigma_{ij} \in C, i < j\}.$$

Definition. C is **good** if $S(C) \cap L(C) = \emptyset$.

How this compares to the previous definition

This is the same condition as before — the original said "for any $\sigma_{ij} \in C$ with $i < j$ there is no $\sigma_{ki} \in C$ with $k < i$ ", which is exactly "no element is both a small endpoint of some transposition and a large endpoint of another," i.e., $S(C) \cap L(C) = \emptyset$. The new phrasing is cleaner: you extract the two sets directly and ask for disjointness.

Connection to the tree

Notice the notation: these are the same S and L that appear in the tree states (L, S, π) . By the invariant theorem, at any node of T_n :

$$L = \text{aex}(\pi), \quad S = \text{exc}(\pi),$$

and these are automatically disjoint (an element can't simultaneously satisfy $\pi(i) < i$ and $\pi(i) > i$). So every tree state has a "good" pair (L, S) in this sense.

Moreover, tracing a path from the root $(\emptyset, \emptyset, \text{id})$ to a node, each step appends a transposition $\tau_{x,y}$ (with $x < y$ forced by the rule) and adjoins x to S and y to L . So the collection C of transpositions used along any root-to-node path is a good collection — $S(C)$ and $L(C)$ are literally the S and L at the endpoint, which are disjoint.

Ready for how good collections and increasing representations relate?

4/14/2026, 2:36:44 PM



A good representation of a permutation is a composition of transpositions that form a good collection

4/14/2026, 2:38:08 PM



Formalization

Definition. Let $\pi \in S_n$. A **good representation** of π is a tuple $(\sigma_1, \sigma_2, \dots, \sigma_m)$ of transpositions in S_n (with $m \geq 0$) such that:

1. $\pi = \sigma_1 \circ \sigma_2 \circ \dots \circ \sigma_m$, and
2. the underlying set $C := \{\sigma_1, \sigma_2, \dots, \sigma_m\}$ is a good collection, i.e., $S(C) \cap L(C) = \emptyset$.

Implicit in (2) is that the σ_i are **distinct** — otherwise "the underlying set" has fewer than m elements, and we should say so. I'll assume distinctness is part of the definition (since a good collection is a set of distinct transpositions); flag it if you meant to allow repeats.

Unpacking the condition

Write each $\sigma_k = \tau_{i_k, j_k}$ with $i_k < j_k$. Condition (2) says

$$\{i_1, i_2, \dots, i_m\} \cap \{j_1, j_2, \dots, j_m\} = \emptyset,$$

i.e., no element of $[n]$ appears as a small endpoint in one transposition and a large endpoint in another. Together with (1), this is a strong constraint: you're factoring π into transpositions whose endpoints, taken collectively, split cleanly into a "small side" S and a "large side" L .

Some immediate consequences:

- **Identity.** The empty tuple is a good representation of id .
- **Length bound.** $m \leq \lfloor n/2 \rfloor \cdot \lceil n/2 \rceil$, since each σ_k is an edge between S and L in a bipartite graph on $[n]$ — but this bound is loose. A tighter observation: in particular $m \leq \binom{n}{2}$ trivially.

- **No "walks."** You cannot have $\tau_{a,b}$ and $\tau_{b,c}$ both in the collection with $a < b < c$: b would be a large endpoint in the first and a small endpoint in the second.
- **Support sizes.** $|S(C)| + |L(C)| \leq n$, with each $|S(C)|, |L(C)| \leq m$.

Examples

Take $n = 5$.

- $\pi = \tau_{1,3} \circ \tau_{2,4}$. Collection $\{\tau_{1,3}, \tau_{2,4}\}$. $S = \{1, 2\}$, $L = \{3, 4\}$. Disjoint ✓ — **good representation**.
- $\pi = \tau_{1,2} \circ \tau_{2,3}$. This equals the 3-cycle $(1\ 2\ 3)$. $S = \{1, 2\}$, $L = \{2, 3\}$. Element 2 conflicts — **not good**. (Consistent with the general fact that a 3-cycle has no good representation, because any factorization into two transpositions with distinct supports must share an endpoint.)
- The product $\tau_{1,4} \circ \tau_{2,4} \circ \tau_{3,4}$ (a "star" into **4\$**). $S = \{1, 2, 3\}$, $L = \{4\}$ — disjoint, so this is a good representation of whatever permutation it computes. (It's a 4-cycle involving 1, 2, 3, 4.)

Connection to the tree, continued

From the previous message: every root-to-node path in T_n produces a good collection. In fact it produces a good **representation** of the node's π : reading the transpositions $\tau_{x_1, y_1}, \tau_{x_2, y_2}, \dots, \tau_{x_m, y_m}$ applied at depths $1, 2, \dots, m$, we have

$$\pi = \tau_{x_1, y_1} \circ \tau_{x_2, y_2} \circ \dots \circ \tau_{x_m, y_m},$$

and the collection is good with $S = \backslash \text{exc}(\pi)$, $L = \backslash \text{aex}(\pi)$.

So **every** $\pi \in S_n$ **has at least one good representation** — namely the one read off its path in T_n . The converse (that every good representation corresponds to a path) would be more delicate, since the tree constrains the *order* in which transpositions can be applied, not just their set.

Is the next move to connect "good" to "increasing"?

4/14/2026, 2:38:30 PM



show that every path in the tree gives a good and increasing representation

4/14/2026, 2:38:54 PM



Ran command ▾

Tool: bash_tool

Done

Transpositions along paths are always distinct. Here's the proof, written up as a short lemma tacked onto the main result.

Created file, ran command, presented file ▾

short LaTeX note proving paths give good increasing reps

Tool: bash_tool

Presenting file(s)...

Done

Three pieces, two of them one-liners given everything we already have:

(A) Product equals π . Immediate from the definition $\pi_k = \pi_{k-1} \circ \tau_{x_k, y_k}$.

(B) Strictly increasing length. The rule forces $\pi_{k-1}(x_k) < \pi_{k-1}(y_k)$ with $x_k < y_k$, so by the transposition-inversion lemma, $\ell(\pi_k) > \ell(\pi_{k-1})$.

(C) Good. The collection C has $S(C) \subseteq S_m$ and $L(C) \subseteq L_m$, which are disjoint because $\backslash \text{exc}(\pi) \cap \backslash \text{aex}(\pi) = \emptyset$.

Distinctness lemma (needed for (C) to give $|C| = m$). This is the only non-trivial piece. The argument: once x enters S at step k it stays there, so any intermediate step touching position x must have x as the small endpoint (since putting x as a large endpoint would require $x \notin S$). That step *increases* $\pi_i(x)$. Symmetrically intermediate steps touching y decrease $\pi_i(y)$. Chaining:

$$\pi_{j-1}(y) \leq \pi_k(y) = \pi_{k-1}(x) < \pi_{k-1}(y) = \pi_k(x) \leq \pi_{j-1}(x),$$

which contradicts the step- j precondition $\pi_{j-1}(x) < \pi_{j-1}(y)$.

I verified distinctness computationally for $n \leq 6$ (all 720 paths of T_6 use distinct transpositions).



path
Pdf



path
Tex



4/14/2026, 2:41:27 PM